ECE 3849 Real-Time Embedded Systems

Lab 2: Improved Oscilloscope

Rachel Turman, Tracy Yang

Signoff TA and Date: Eric 4/10/25

Introduction:

In this lab, the objective is to improve the digital oscilloscope that was made in the previous lab through the use of adding triggers, scaling, and displaying the CPU load. This was done by developing a real-time application. The application handles the processes without the use of an operating system and instead uses interrupt prioritization and preemption. From this, lab group was able to successfully access data safely with local variables to prevent data loss and incorrect data presentation, meet tight timing constraints within the workspace, write performance-sensitive code, and use real-time debugging techniques.

Discussion & Results:

The requirements for improving the oscilloscope were the following: 1) displaying at a high frame rate a waveform at 20us per division and 1V per division with rising edge trigger, 2) programming a selectable trigger slope, 3) developing an adjustable voltage scale of 2V, 1V, 500mV, 200mV, and 100mV, 4) calculating and displaying the CPU load of the ISRs, and 5) pass button inputs through a FIFO without shared data bugs.

To begin the lab, the lab group first created the FIFO for the button inputs. By modifying the current code to take the button inputs into a FIFO data structure, it ensures that all button presses are not lost, especially for operations that may take longer to complete. The FIFO was implemented through the functions fifo_put() and fifo_get(). Within buttons.c, instead of the functionality of the buttons being coded there, it was replaced with fifo_put(), and the button presses corresponded to the location of where in the FIFO it is being placed in. In main(), a local FIFO was made in case there were interrupts in the middle of reading button values. fifo_get() called for which buttons were being pressed and did the corresponding button functionalities. With all this, the implementation of a FIFO ensures that there are no shared data bugs *without* disabling interrupts.

Once completed, the next thing to design is the trigger search for the waveform. The trigger was put in the Snapshot() function with a search count. Once it reached the max edge search, it would either draw the waveform as a flat signal, the rising edge, or the falling edge. This also needed to be done while ensuring there were no shared data bugs for the ADC buffer. While the group had trouble with this (the waveform would shake because of shared data bugs), eventually, they were able to resolve this error with the help of the TA. Lots of debugging was necessary for this step as many things were accessing gADCBuffer, especially with the next task.

Another issue the team had was changing the waveform's scale. There was much confusion about having the correct voltage measurements displayed on the LCD screen. There were issues with converting the ADC values to the correct voltage while still aligning with the oscilloscope's grid. An example of this was incorrectly scaling off by a factor of 2. Another included the scaling being correct, but the offsets were slightly off. After trial and error, the team successfully displayed and offset the waveform to remain in the middle of the Y-axis while graphing the maximum

voltage on the screen. This was through the use of a switch case containing the values of the scaling specified and connecting the states with a button as well as a mathematical function that maintained the scaling for the LCD screen.

The final implementation was the CPU load measurement. This was one of the more straightforward tasks after a quick discussion with the TA. A timer was set in the beginning of main() to initialize the unloaded load. Then, in the while loop where all the interrupts and functions are called, the timer was reset and ran to determine the time it took to start and complete tasks.

Conclusion:

Lab 2 showed how imperative handling shared data and meeting time constraints were in real-time operations. The functions and variables were intentionally created and accessed throughout the code. Local variables allowed data manipulation. Displaying the scale, CPU load, and trigger direction allowed for simpler debugging. The FIFO allowed for data to maintain its integrity and accuracy. It was through these considerations that the team was able to successfully make improvements to the digital oscilloscope.